

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_2be29ecf-a31\agent-ad22887c34408cc57.jsonl`  
- SHA-256: `90215e54ad79ae5312fd705b7902ed273f956309dd4eac0e3f48cf2f10626c17`  
- Source modified: `2026-06-22T01:31:49+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_2be29ecf-a31`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T01:30:51.585Z)

Adversarially verify this P2b-install review finding. Read the cited code in C:\Users\avidu\Projects\bhi-p2b yourself (run `cd C:\Users\avidu\Projects\bhi-p2b && git diff origin/master...HEAD` and open the file). Decide if it is a REAL defect that should block or be fixed, or a false alarm / nit. Default to is_real=false if it does not hold up.

Lens: plan-consistency-regression

Severity(claimed): nit

Title: install.py/uninstall.py IO orchestration (main) exercised only via --dry-run, not asserted in tests

File: C:\Users\avidu\Projects\bhi-p2b\tests\test_p2b_install.py

Detail: The new tests cover the pure helpers (shortcut_spec, resolve_method, install_command, build_manifest, uninstall_command, find_manifest) plus a manifest roundtrip ? these are substantive and non-tautological. The IO/orchestration bodies of main() (writing the shortcut, chmod, copying the uninstaller, _rm tree-walk, the --purge empty-dir cleanup) are not directly tested. I confirmed `python scripts/install.py --dry-run` runs cleanly and resolves the Windows app-home correctly, but the real-write path and uninstall.py's _rm/--purge logic have no coverage. This is consistent with the repo convention of testing pure-logic locally and handing machine-side install verification to the owner (a clean-VM install?reel?uninstall is an explicit DoD item in PACKAGING_PLAN.md §9), so it is a nit, not a gap to block on.

Suggested fix: Optional follow-up: a tmp_path-scoped test that calls install.main() against a fake app-home (monkeypatch default_os_app_home + stub _run) and then uninstall.main() to assert the shortcut/manifest are created then removed while user_data is kept (and deleted on --purge). Not required for this PR.

2. assistant (2026-06-22T01:30:54.105Z)

I'll verify this finding by examining the actual code in the cited repository.

3. user (2026-06-22T01:30:57.266Z)

78e423d docs(pkg): link P2b-install PR #310 in the §7 tracker

a5c1493 feat(pkg): P2b-install ? batteries-included install + truly-local 'motion' first-run default (gate 08)

4. user (2026-06-22T01:30:57.437Z)

```
README.md | 31 ++++++  
backend/config/__init__.py | 9 +-  
backend/config/loader.py | 34 ++++++  
backend/main.py | 15 ++++  
docs/PACKAGING_PLAN.md | 13 +-  
scripts/install.py | 210 ++++++  
scripts/uninstall.py | 150 ++++++  
tests/test_app_launcher.py | 42 ++++++
```

```
tests/test_p2b_install.py | 163 ++++++
9 files changed, 660 insertions(+), 7 deletions(-)
```

5. assistant (2026-06-22T01:30:59.699Z)

Let me read the test file and the install/uninstall scripts.

6. user (2026-06-22T01:31:00.656Z)

```
1      """P2b-install (PACKAGING_PLAN.md): batteries-included config seeding +
install/uninstall tooling.
2
3      Covers:
4      * the engine-side ``write_light_default_config`` (gate 08 default = truly-local +
motion), incl.
5      the load-bearing **no-overwrite** idempotency and the EFFECTIVE config the loader
produces;
6      * the pure logic of ``scripts/install.py`` / ``scripts/uninstall.py`` (shortcut
content, manifest,
7      installer/uninstaller argv, method resolution, OS app-home) ? no real subprocess/I/O.
8      """
9      from __future__ import annotations
10
11     import importlib.util
12     import json
13     from pathlib import Path
14
15     import pytest
16
17     from backend.config import LIGHT_DEFAULT_CONFIG, load_config, write_light_default_config
18
19
20     def _load_script(name: str):
21         """Load a scripts/*.py bootstrap file as a module (it is stdlib-only, safe to
exec)."""
22         path = Path(__file__).resolve().parents[1] / "scripts" / name
23         spec = importlib.util.spec_from_file_location(name[:-3], path)
24         assert spec and spec.loader
25         mod = importlib.util.module_from_spec(spec)
26         spec.loader.exec_module(mod)
27         return mod
28
29
30     # --- engine seeding: write_light_default_config (gate 08)
-----
31
32     def test_seed_writes_truly_local_motion(tmp_path):
33         cfg = tmp_path / "sub" / "config.json" # also proves parent-dir creation
34         path, written = write_light_default_config(cfg)
35         assert written is True and path == cfg and cfg.exists()
36         data = json.loads(cfg.read_text(encoding="utf-8"))
37         assert data == LIGHT_DEFAULT_CONFIG
38         assert data["indexing"]["default_segmenter"] == "motion"
39         assert data["indexing"]["use_gpu"] is False
40         assert data["deployment"]["profile"] == "truly-local"
41
42
43     def test_seed_does_not_overwrite_existing(tmp_path):
44         cfg = tmp_path / "config.json"
45         cfg.write_text('{"sport": "tennis", "indexing": {"default_segmenter": "gemini"}}',
encoding="utf-8")
46         path, written = write_light_default_config(cfg)
47         assert written is False and path == cfg
48         # The user's file is preserved untouched (the wizard/edits are never clobbered).
49         assert json.loads(cfg.read_text(encoding="utf-8"))["indexing"]["default_segmenter"]
```

```

== "gemini"
50
51
52     def test_seed_overwrite_true_forces(tmp_path):
53         cfg = tmp_path / "config.json"
54         cfg.write_text("{} ", encoding="utf-8")
55         _, written = write_light_default_config(cfg, overwrite=True)
56         assert written is True
57         assert json.loads(cfg.read_text(encoding="utf-8")) == LIGHT_DEFAULT_CONFIG
58
59
60     def test_seeded_config_loads_to_truly_local_motion(tmp_path):
61         """The minimal seeded file, run through the loader, expands via the truly-local
PRESET to a
62         coherent torch-free/offline config (motion + no Gemini handoff + local storage + no
GPU)."""
63         cfg = tmp_path / "config.json"
64         write_light_default_config(cfg)
65         loaded = load_config(cfg)
66         assert loaded.indexing.default_segmenter == "motion"
67         assert loaded.indexing.use_gpu is False
68         assert loaded.deployment.profile == "truly-local"
69         assert loaded.indexing.skip_ai_handoff is True          # supplied by the truly-local
preset
70         assert loaded.storage.backend == "local"                # supplied by the truly-local
preset
71
72
73     # --- scripts/install.py pure logic
-----
74
75     @pytest.fixture(scope="module")
76     def install_mod():
77         return _load_script("install.py")
78
79
80     @pytest.fixture(scope="module")
81     def uninstall_mod():
82         return _load_script("uninstall.py")
83
84
85     @pytest.mark.parametrize(
86         "system, fname, executable",
87         [
88             ("Windows", "KhelSutra.bat", False),
89             ("Darwin", "KhelSutra.command", True),
90             ("Linux", "khelsutra.sh", True),
91         ],
92     )
93     def test_shortcut_spec_per_os(install_mod, system, fname, executable):
94         home = Path("/home/u/AppHome")
95         name, content, ex = install_mod.shortcut_spec(system, home)
96         assert name == fname and ex is executable
97         # str(home) is OS-native (backslashes on Windows) ? assert against it, not a
hardcoded literal.
98         assert "RALLY_APP_HOME" in content and str(home) in content
99         assert "indexer --app" in content
100
101     def test_resolve_method(install_mod):
102         assert install_mod.resolve_method("auto", uv_present=True) == "uv"
103         assert install_mod.resolve_method("auto", uv_present=False) == "pip"
104         assert install_mod.resolve_method("pip", uv_present=True) == "pip"    # explicit wins
105         assert install_mod.resolve_method("uv", uv_present=False) == "uv"
106
107     def test_install_command_argv(install_mod):
108         assert install_mod.install_command("uv", ".") == ["uv", "tool", "install", "."]

```

```

108     pip = install_mod.install_command("pip", "badminton-highlight-indexer")
109     assert pip[1:] == ["-m", "pip", "install", "badminton-highlight-indexer"]
110
111
112     def test_default_os_app_home_per_os(install_mod):
113         win = install_mod.default_os_app_home(system="Windows", env={"APPDATA":
114 r"C:\Users\u\AppData\Roaming"})
115         assert win.name == "Khelsutra" and "Roaming" in str(win)
116         lin = install_mod.default_os_app_home(system="Linux", env={"XDG_DATA_HOME":
117 "/home/u/.local/share"})
118         assert str(lin) == str(Path("/home/u/.local/share") / "Khelsutra")
119
120     def test_build_manifest_shape(install_mod):
121         home = Path("/home/u/AppHome")
122         m = install_mod.build_manifest(
123             method="uv", app_home=home, shortcut=home / "khelsutra.sh",
124             created=[home / "khelsutra.sh", home / "uninstall_manifest.json"],
125             timestamp="2026-06-22T00:00:00+00:00",
126         )
127         assert m["package"] == "badminton-highlight-indexer"
128         assert m["install_method"] == "uv"
129         assert str(home / "config.json") in m["user_data"] # user data is tracked
130 separately (kept on uninstall)
131         assert m["shortcut"] == str(home / "khelsutra.sh")
132         assert m["schema"] == 1
133
134     # --- scripts/uninstall.py pure logic
135     -----
136
137     def test_uninstall_command_argv(uninstall_mod):
138         assert uninstall_mod.uninstall_command("uv", "badminton-highlight-indexer") == \
139             ["uv", "tool", "uninstall", "badminton-highlight-indexer"]
140         pip = uninstall_mod.uninstall_command("pip", "badminton-highlight-indexer")
141         assert pip[1:] == ["-m", "pip", "uninstall", "-y", "badminton-highlight-indexer"]
142
143     def test_find_manifest_explicit(uninstall_mod, tmp_path):
144         m = tmp_path / "uninstall_manifest.json"
145         m.write_text("{} ", encoding="utf-8")
146         assert uninstall_mod.find_manifest(str(m)) == m
147         assert uninstall_mod.find_manifest(str(tmp_path / "nope.json")) is None
148
149     def test_install_uninstall_roundtrip_manifest(install_mod, uninstall_mod, tmp_path):
150         """The manifest install.py would write is consumable by uninstall.py's
151 expectations."""
152         home = tmp_path / "AppHome"
153         m = install_mod.build_manifest(
154             method="pip", app_home=home, shortcut=home / "KhelSutra.bat",
155             created=[home / "KhelSutra.bat", home / "uninstall_manifest.json"],
156             timestamp="2026-06-22T00:00:00+00:00",
157         )
158         mp = home / "uninstall_manifest.json"
159         home.mkdir(parents=True)
160         mp.write_text(json.dumps(m), encoding="utf-8")
161         assert uninstall_mod.find_manifest(str(mp)) == mp
162         loaded = json.loads(mp.read_text(encoding="utf-8"))
163         assert uninstall_mod.uninstall_command(loaded["install_method"],
164 loaded["package"])[0:3] in (
165             ["uv", "tool", "uninstall"], [install_mod.sys.executable, "-m", "pip"],
166         )

```

7. user (2026-06-22T01:31:01.137Z)

```
1      #!/usr/bin/env python3
2      """Batteries-included installer for the KhelSutra highlight indexer (PACKAGING_PLAN.md
3      P2b).
4      Cross-platform (Windows / macOS / Linux), no native component (01):
5
6      * installs the LIGHT (torch-free) package via ``uv tool install`` (preferred ? an
isolated tool
7      env that puts ``indexer`` on PATH) or ``pip install`` (fallback);
8      * creates a per-OS launcher shortcut that pins ``RALLY_APP_HOME`` to the OS app-data
dir and runs
9      ``indexer --app`` ? so config / .env / DB / reels live in ONE stable place
10     (mirrors ``backend.utils.app_home.default_os_app_home``). Pinning the env BEFORE the
process
11     starts is load-bearing: the engine's ``load_dotenv`` runs at import, so only an env
set by the
12     launcher (not by the engine) roots the ``.env`` in the app-home;
13     * writes an uninstall manifest so ``scripts/uninstall.py`` can cleanly reverse it.
14
15     The truly-local + ``motion`` default config is seeded by the ENGINE itself on the first
16     ``indexer --app`` run (``write_light_default_config``), so it works regardless of
install method;
17     this script only wires the install + the launcher.
18
19     Usage::
20
21         python scripts/install.py                # install '.' (this checkout), uv-
if-present else pip
22         python scripts/install.py --target badminton-highlight-indexer  # from PyPI (once
published)
23         python scripts/install.py --method pip      # force pip
24         python scripts/install.py --dry-run         # print the plan, change nothing
25
26     stdlib-only so it runs as a standalone bootstrap BEFORE the package exists.
27     """
28     from __future__ import annotations
29
30     import argparse
31     import json
32     import os
33     import platform
34     import shutil
35     import subprocess
36     import sys
37     from datetime import datetime, timezone
38     from pathlib import Path
39
40     PACKAGE = "badminton-highlight-indexer"
41     APP_DIR_NAME = "Khelsutra" # mirrors backend.utils.app_home.APP_DIR_NAME
42     MANIFEST_NAME = "uninstall_manifest.json"
43
44
45     # --- pure helpers (unit-tested)
-----
46
47     def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
48                             env: dict[str, str] | None = None) -> Path:
49         """The OS-conventional per-user app-data dir ? a standalone mirror of
50         ``backend.utils.app_home.default_os_app_home`` (this script runs before the package
exists)."""
51         system = system or platform.system()
52         e = os.environ if env is None else env
53         if system == "Windows":
```

```

54         base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
55     elif system == "Darwin":
56         base = str(Path.home() / "Library" / "Application Support")
57     else:
58         base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
59     return Path(base) / app_name
60
61
62     def shortcut_spec(system: str, app_home: Path, indexer_cmd: str = "indexer") ->
tuple[str, str, bool]:
63         """Return `(filename, content, make_executable)` for the per-OS launcher shortcut.
64
65         The shortcut pins `RALLY_APP_HOME` then runs `indexer --app`. PURE (no IO) so
it's testable.
66         """
67         home = str(app_home)
68         if system == "Windows":
69             content = (
70                 "@echo off\r\n"
71                 "rem KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens
the app.\r\n"
72                 f'set "RALLY_APP_HOME={home}"\r\n'
73                 f'{indexer_cmd} --app\r\n"
74             )
75             return "KhelSutra.bat", content, False
76         if system == "Darwin":
77             content = (
78                 "#!/bin/bash\n"
79                 "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
80                 f'export RALLY_APP_HOME="{home}"\n'
81                 f"exec {indexer_cmd} --app\n"
82             )
83             return "KhelSutra.command", content, True
84         # Linux / other
85         content = (
86             "#!/bin/bash\n"
87             "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
88             f'export RALLY_APP_HOME="{home}"\n'
89             f"exec {indexer_cmd} --app\n"
90         )
91         return "KhelSutra.sh", content, True
92
93
94     def build_manifest(*, method: str, app_home: Path, shortcut: Path, created: list[Path],
95                       timestamp: str) -> dict:
96         """The uninstall manifest (PURE). `created` = files THIS installer made (safe to
remove);
97         `user_data` = paths the engine/user own (kept unless `uninstall.py --purge`)."""
98         return {
99             "schema": 1,
100             "product": "KhelSutra highlight indexer",
101             "package": PACKAGE,
102             "install_method": method,          # "uv" | "pip"
103             "indexer_command": "indexer",
104             "app_home": str(app_home),
105             "shortcut": str(shortcut),
106             "created": [str(p) for p in created],
107             "user_data": [
108                 str(app_home / "config.json"),
109                 str(app_home / ".env"),
110                 str(app_home / "output"),
111             ],
112             "timestamp": timestamp,

```

```

113     }
114
115
116     def resolve_method(requested: str, uv_present: bool) -> str:
117         """Pick the installer: explicit beats auto; auto prefers uv when present, else
118         pip."""
119         if requested in ("uv", "pip"):
120             return requested
121         return "uv" if uv_present else "pip"
122
123     def install_command(method: str, target: str) -> list[str]:
124         """The argv for the chosen installer (PURE)."""
125         if method == "uv":
126             return ["uv", "tool", "install", target]
127         return [sys.executable, "-m", "pip", "install", target]
128
129
130     # --- IO / orchestration
131     -----
132
133     def _run(argv: list[str]) -> None:
134         print(f" $ {' '.join(argv)}")
135         subprocess.run(argv, check=True)
136
137     def main(argv: list[str] | None = None) -> int:
138         ap = argparse.ArgumentParser(description="Install the KhelSutra highlight indexer
139         (LIGHT tier).")
140         ap.add_argument("--target", default=".",
141             help="What to install: '.' (this checkout, default) or a PyPI
142             name/spec.")
143         ap.add_argument("--method", choices=("auto", "uv", "pip"), default="auto",
144             help="Installer to use (default: uv if present, else pip).")
145         ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
146             nothing.")
147         args = ap.parse_args(argv)
148
149         system = platform.system()
150         app_home = default_os_app_home()
151         method = resolve_method(args.method, shutil.which("uv") is not None)
152         cmd = install_command(method, args.target)
153         fname, content, executable = shortcut_spec(system, app_home)
154         shortcut = app_home / fname
155
156         print(f"KhelSutra installer ? {PACKAGE} (LIGHT, torch-free)")
157         print(f" OS          : {system}")
158         print(f" installer   : {method}")
159         print(f" target      : {args.target}")
160         print(f" app-home    : {app_home}")
161         print(f" shortcut    : {shortcut}")
162         if args.dry_run:
163             print("\n[dry-run] would run:")
164             print(f" $ {' '.join(cmd)}")
165             print(f" write shortcut -> {shortcut}")
166             print(f" write manifest -> {app_home / MANIFEST_NAME}")
167             print(f" copy uninstaller -> {app_home / 'uninstall.py'}")
168             return 0
169
170         print("\n[1/3] installing the package?")
171         try:
172             _run(cmd)
173         except (subprocess.CalledProcessError, FileNotFoundError) as e:
174             print(f"[ERROR] install failed: {e}")
175             if method == "uv":

```

```

173         print(" (uv not usable? re-run with --method pip)")
174     return 1
175
176     print("\n[2/3] writing the launcher shortcut + app-home?")
177     app_home.mkdir(parents=True, exist_ok=True)
178     shortcut.write_text(content, encoding="utf-8")
179     if executable:
180         shortcut.chmod(0o755)
181     # Make the uninstaller available next to the data it removes.
182     created: list[Path] = [shortcut]
183     here = Path(__file__).resolve().parent
184     src_uninstall = here / "uninstall.py"
185     dst_uninstall = app_home / "uninstall.py"
186     if src_uninstall.exists():
187         shutil.copyfile(src_uninstall, dst_uninstall)
188         created.append(dst_uninstall)
189
190     print("\n[3/3] writing the uninstall manifest?")
191     manifest_path = app_home / MANIFEST_NAME
192     created.append(manifest_path)
193     manifest = build_manifest(
194         method=method, app_home=app_home, shortcut=shortcut, created=created,
195         timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
196     )
197     manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")
198
199     print("\n? Installed. To launch KhelSutra:")
200     if system == "Windows":
201         print(f"    double-click {shortcut}    (or run `indexer --app`)")
202     else:
203         print(f"    {shortcut}    (or run `indexer --app`)")
204     print("    The first run seeds a truly-local 'motion' config (offline, no key/GPU
needed).")
205     print(f"    To uninstall: python \"{dst_uninstall}\"    (add --purge to also delete
your data)")
206     return 0
207
208
209 if __name__ == "__main__":
210     raise SystemExit(main())
211

```

8. user (2026-06-22T01:31:01.612Z)

```

1     #!/usr/bin/env python3
2     """Clean uninstaller for the KhelSutra highlight indexer (PACKAGING_PLAN.md P2b).
3
4     Reverses ``scripts/install.py`` from the manifest it wrote:
5
6     * uninstalls the package (``uv tool uninstall`` / ``pip uninstall -y`` ? matches the
install method);
7     * removes the files the installer created (launcher shortcut, the copied uninstaller,
the manifest);
8     * by default **KEEPS your data** (the seeded ``config.json``, your ``.env`` with the
Gemini key,
9     and the ``output/`` reels + DB). Pass ``--purge`` to delete those too.
10
11     Usage::
12
13         python uninstall.py                # find the manifest in the app-home, keep user
data
14         python uninstall.py --purge        # also delete config.json / .env / output (your
data)
15         python uninstall.py --manifest /path/to/uninstall_manifest.json
16         python uninstall.py --dry-run      # print the plan, change nothing

```



```

17
18     stdlib-only; safe to run from the copy the installer dropped in the app-home.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import json
24     import os
25     import platform
26     import shutil
27     import subprocess
28     import sys
29     from pathlib import Path
30
31     APP_DIR_NAME = "Khelsutra"
32     MANIFEST_NAME = "uninstall_manifest.json"
33
34
35     def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
36                             env: dict[str, str] | None = None) -> Path:
37         """Mirror of the installer's app-home resolver (used to LOCATE the manifest when not
38         given)."""
39         system = system or platform.system()
40         e = os.environ if env is None else env
41         if system == "Windows":
42             base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
43         elif system == "Darwin":
44             base = str(Path.home() / "Library" / "Application Support")
45         else:
46             base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
47         return Path(base) / app_name
48
49     def find_manifest(explicit: str | None) -> Path | None:
50         """The manifest path: explicit arg, else next to THIS file, else the default app-
51         home."""
52         if explicit:
53             p = Path(explicit)
54             return p if p.exists() else None
55         for cand in (Path(__file__).resolve().parent / MANIFEST_NAME,
56                     default_os_app_home() / MANIFEST_NAME):
57             if cand.exists():
58                 return cand
59         return None
60
61     def uninstall_command(method: str, package: str) -> list[str]:
62         """The argv to remove the package (PURE)."""
63         if method == "uv":
64             return ["uv", "tool", "uninstall", package]
65         return [sys.executable, "-m", "pip", "uninstall", "-y", package]
66
67
68     def _rm(path: Path, dry: bool) -> None:
69         try:
70             if path.is_dir() and not path.is_symlink():
71                 print(f" - rmtree {path}")
72                 if not dry:
73                     shutil.rmtree(path, ignore_errors=True)
74             elif path.exists() or path.is_symlink():
75                 print(f" - remove {path}")
76                 if not dry:
77                     path.unlink(missing_ok=True)
78         except OSError as e:
79             print(f" ! could not remove {path}: {e}")

```

```

80
81
82     def main(argv: list[str] | None = None) -> int:
83         ap = argparse.ArgumentParser(description="Uninstall the KhelSutra highlight
indexer.")
84         ap.add_argument("--manifest", default=None, help="Path to uninstall_manifest.json.")
85         ap.add_argument("--purge", action="store_true",
86                         help="Also delete YOUR data (config.json, .env, output/). Default
keeps it.")
87         ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
nothing.")
88         args = ap.parse_args(argv)
89
90         manifest_path = find_manifest(args.manifest)
91         if not manifest_path:
92             print("[ERROR] no uninstall_manifest.json found (pass --manifest <path>).")
93             return 1
94         manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
95         method = manifest.get("install_method", "pip")
96         package = manifest.get("package", "badminton-highlight-indexer")
97         created = [Path(p) for p in manifest.get("created", [])]
98         user_data = [Path(p) for p in manifest.get("user_data", [])]
99         app_home = Path(manifest.get("app_home", str(manifest_path.parent)))
100
101         print(f"KhelSutra uninstaller ? {package} (installed via {method})")
102         print(f"  manifest : {manifest_path}")
103         print(f"  app-home : {app_home}")
104         print(f"  data      : {'WILL BE DELETED (--purge)' if args.purge else 'kept (use
--purge to delete)'}")
105         dry = args.dry_run
106
107         print("\n[1/3] removing the package?")
108         cmd = uninstall_command(method, package)
109         print(f"  $ {' '.join(cmd)}")
110         if not dry:
111             try:
112                 subprocess.run(cmd, check=False)
113             except FileNotFoundError as e:
114                 print(f"  ! {method} not found ({e}); remove the package manually.")
115
116         print("\n[2/3] removing installer-created files?")
117         # Remove the manifest LAST (we still reference it); skip the running uninstaller
copy itself.
118         self_path = Path(__file__).resolve()
119         for p in created:
120             if p.resolve() == self_path or p.resolve() == manifest_path.resolve():
121                 continue
122             _rm(p, dry)
123
124         if args.purge:
125             print("\n[3/3] purging user data (--purge)?")
126             for p in user_data:
127                 _rm(p, dry)
128         else:
129             print("\n[3/3] keeping your data:")
130             for p in user_data:
131                 print(f"  · {p}")
132
133         # Manifest + (optionally) the empty app-home last.
134         _rm(manifest_path, dry)
135         if args.purge and app_home.exists():
136             try:
137                 remaining = [c for c in app_home.iterdir() if c.resolve() != self_path]
138                 if not remaining:
139                     _rm(app_home, dry)

```

```

140         except OSError:
141             pass
142
143         print("\n? Uninstalled." + (" " if args.purge else " Your data was kept. "))
144         if self_path.parent == app_home:
145             print(f"    (you can delete this script + the app-home folder manually:
{app_home})")
146         return 0
147
148
149     if __name__ == "__main__":
150         raise SystemExit(main())
151

```

9. assistant (2026-06-22T01:31:07.155Z)

The finding's factual claims hold up. Let me verify the test execution to confirm the tests actually run and that `main()` orchestration is genuinely untested. Let me also check whether there is any other test file that might cover `main()`.

10. user (2026-06-22T01:31:09.103Z)

<persisted-output>

Output too large (30.5KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\tool-results\toolu_01Thk95AHPrcGAqRjHo6HdVR.txt

Preview (first 2KB):

```

C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:35: def env(tmp_path, monkeypatch):
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:38:     monkeypatch.setattr(m,
"db_instance", db)
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:39:     monkeypatch.setattr(m,
"global_config", cfg)
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:40:     monkeypatch.setattr(m,
"_get_executor", lambda: _InlineExecutor())
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:42:     return client, db, tmp_path,
monkeypatch
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:120: def
test_parity_job_model_with_direct_processing(env, tmp_path, monkeypatch):
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:122:     (submit + _InlineExecutor
leading to _run_claimed_job / drain, plus explicit _run_claimed_job)
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:140:     # Job-worker path via public
API submit (exercises job creation + inline executor + internal _run_claimed_job)
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:147:     # Explicit job-worker via
_run_claimed_job (create + mark running + drive the claimed runner)
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:152:     db.mark_job_running(jid)
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:154:     m._run_claimed_job(job_row)
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:156:     assert len(segs2) >= 1,
f"_run_claimed_job path must produce segments for vid {vid2}"
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:314: def
test_fail_stale_jobs_terminalizes_queued_and_running_only(env):
C:\Users\avidu\Projects\bhi-p2b\tests\test_async_jobs.py:318:     db.mark_job_running("r1")
C:\Users\avidu\Projects\bhi-p2b\tests\test_app_home.py:17: def _clear_app_home_env(monkeypatch):
C:\Users\avidu\Projects\bhi-p2b\tests\test_app_home.py:19:
monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)
...
</persisted-output>

```

11. user (2026-06-22T01:31:11.145Z)

..... [100%]
14 passed in 0.29s

12. assistant (2026-06-22T01:31:14.140Z)

Let me confirm the grep results specifically for the install/uninstall test file and check whether `main()` is referenced anywhere in tests.

13. user (2026-06-22T01:31:15.536Z)

No matches found

14. assistant (2026-06-22T01:31:18.589Z)

That confirms it: the test file does not reference `main`, `monkeypatch`, `\_run`, `\_rm`, `chmod`, `copyfile`, or `rmtree` at all. Let me verify the dry-run claim and check the PACKAGING_PLAN DoD reference the finding cites.

15. user (2026-06-22T01:31:20.443Z)

KhelSutra installer ? badminton-highlight-indexer (LIGHT, torch-free)

```
OS      : Windows
installer : pip
target   : .
app-home : C:\Users\avidu\AppData\Roaming\Khelsutra
shortcut : C:\Users\avidu\AppData\Roaming\Khelsutra\KhelSutra.bat
```

[dry-run] would run:

```
$ C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pip install .
write shortcut -> C:\Users\avidu\AppData\Roaming\Khelsutra\KhelSutra.bat
write manifest -> C:\Users\avidu\AppData\Roaming\Khelsutra\uninstall_manifest.json
copy uninstaller -> C:\Users\avidu\AppData\Roaming\Khelsutra\uninstall.py
```

16. user (2026-06-22T01:31:21.453Z)

15:[Omitted long matching line]

16:2. ? **P2b-install ? DONE (2026-06-22). ** Engine seeds a truly-local + `motion` (gate **08**) torch-free default config on the first `indexer --app` (never overwrites); cross-platform `scripts/install.py` (`uv tool install` else pip) + per-OS launcher shortcut that pins `RALLY\_APP\_HOME` + `scripts/uninstall.py` driven by an install manifest (keeps user data unless `--purge`). See §7.

45:| **L10** | **North star = the FULL quality path is the SOLE completion goal** ? LIGHT is the first increment; DONE requires native WASB + BYO training + in-app annotation | 02, 2026-06-21 | §9 DoD splits "v1 shippable" from "north-star DONE"; P3/P4 (WASB/ONNX/train/annotate) are completion-gating, not optional. |

84:| Gemini API key | Partially | Today `GET /api/config` returns the **whole `AppConfig` unauthenticated** `[verified main.py:144-146]`. **P0** redacts + adds a serialization test + a Host-header/localhost-origin guard (DNS-rebinding vector against `127.0.0.1`). Key lives in app-home `.env` (or OS keychain), never in config; scrubbed on uninstall. |

113:[Omitted long matching line]

115:| P2d | **DB upgrade contract** ? `SCHEMA\_VERSION` + downgrade guard (`SchemaTooNewError`) + backup-before-migrate (`<db>.bak-v<old>`, WAL-complete) + `backup\_database()` export. Byte-identical for fresh/current DBs. *(uninstall manifest deferred to P2b-install.)* | ? | No | ? | [#307](https://github.com/Khelsutra/badminton-highlight-indexer/pull/307) |

127:2. **02 ? RESOLVED 2026-06-21:** v1 = **LIGHT first**, **but the sole completion goal (north star, L10) is the FULL path** ? native WASB + BYO training + in-app annotation. DoD §9 splits "v1 shippable" from "north-star DONE"; P3/P4 are completion-gating, not optional.

136:## 9. Definition of done

140:- [] P2a?P2d merged: a **clean Windows VM ? installer ? reel downloaded (en+hi)** with **no Python/git/ffmpeg pre-installed**; uninstall is clean; upgrade preserves data.

169:- 2026-06-21 ? Owner resolved 01 + 02: **cross-platform (Win/Mac/Linux) webserver, no native component** (L9); **LIGHT-first with a WASB-complete north star as the sole completion goal** (L10). Updated §0/§2/§4.5/§6/§7-P2b/§8/§9 accordingly. Remaining gates: 03 (WASB-C3 counsel), 04 (hosted vs BYO-cloud), 05?09.

170:- 2026-06-22 ? **P3 GPU validation DONE** (§0, L4): the packaged wheel ran end-to-end (torch-free wheel ? `indexer --app` ? bundled SPA ? native-WASB ? 22 rallies ? 73.7 MB reel). **P2b-install DONE**: owner resolved **08 = truly-local + `motion`**; engine seeds it on first `--app` run + `scripts/install.py`/`uninstall.py` + README. §7 P2b ?, §8 08 + §9 08 box resolved. Remaining gates: 03, 04, 05?07, 09.

17. assistant (2026-06-22T01:31:34.778Z)

Every claim in the finding is confirmed. Let me summarize my verification:

****Verified facts:****

1. ****The new tests cover only pure helpers**** ? confirmed by reading `tests/test_p2b_install.py`. The tests exercise: `write_light_default_config` (engine seeding), `shortcut_spec`, `resolve_method`, `install_command`, `default_os_app_home`, `build_manifest`, `uninstall_command`, `find_manifest`, and a manifest roundtrip. All are pure-logic / no real IO.
2. ****`main()` orchestration bodies are untested**** ? confirmed via Grep: the test file contains zero references to `main`, `monkeypatch`, `_run`, `_rm`, `chmod`, `copyfile`, or `rmtree`. The real-write paths in `install.py::main()` (lines 167-206: subprocess `_run`, `mkdir`, `shortcut.write_text`, `chmod`, `shutil.copyfile`, manifest write) and `uninstall.py::main()`/`_rm` (the tree-walk, `--purge` user_data deletion, the empty-app-home cleanup at lines 124-141) have no coverage.
3. ****The dry-run path works cleanly and resolves the Windows app-home correctly**** ? confirmed: `python scripts/install.py --dry-run` outputs `C:\Users\avidu\AppData\Roaming\Khelsutra` and a coherent plan; tests pass (14 passed).
4. ****The repo convention of handing machine-side install verification to the owner is real**** ? confirmed in `docs/PACKAGING_PLAN.md` §9 DoD line 140: "a ****clean Windows VM ? installer ? reel downloaded**** ... uninstall is clean" is an explicit owner-side DoD item. This matches the CLAUDE.md guidance to hand heavy machine-side checks to the owner.

****Assessment:**** The finding is accurate in every particular and is honestly self-limiting. It explicitly classifies itself as a nit (not a gap to block on), correctly reasons that the untested IO orchestration is consistent with the repo's documented convention of testing pure logic locally and deferring clean-VM install/uninstall verification to the owner, and offers an optional (not required) follow-up. This is a legitimate, real observation about test coverage ? it is not fabricated and not a false alarm ? but it is correctly scoped as a nit that should not block. The `--purge` deletion logic is genuinely riskier code that has no automated coverage, which is the one place a follow-up test would add real value, so it is "real" in the sense of being a genuine, non-tautological gap rather than a phantom.

18. user (2026-06-22T01:31:43.318Z)

Structured output provided successfully

19. assistant (2026-06-22T01:31:49.752Z)

I have completed the adversarial verification. The finding is REAL but correctly scoped as a nit (`is_real=true`, `severity=nit`).

All four factual claims in the finding hold up against the actual code in `C:\Users\avidu\Projects\bhi-p2b`:

- The new tests in `tests/test_p2b_install.py` cover only pure helpers ? confirmed by Read and by a Grep showing zero references to `main`, `monkeypatch`, `_run`, `_rm`, `chmod`, `copyfile`, or `rmtree`.
- The IO/orchestration bodies of `install.py::main()` (lines 167-206) and `uninstall.py::main()`/`_rm` (lines 68-141), including the destructive `--purge` user-data deletion, are genuinely untested.
- `python scripts/install.py --dry-run` runs cleanly and resolves the Windows app-home correctly; the 14 tests pass.
- The repo convention of deferring clean-VM install/uninstall verification to the owner is real ? `docs/PACKAGING_PLAN.md` §9 DoD line 140 makes it an explicit owner-side acceptance item.

It is a legitimate, non-tautological coverage gap (not a false alarm), but the finding correctly classifies it as a nit that should not block: the uncovered code is real-write/destructive IO that the repo convention hands to an owner clean-VM run, and the finding only proposes an optional follow-up.